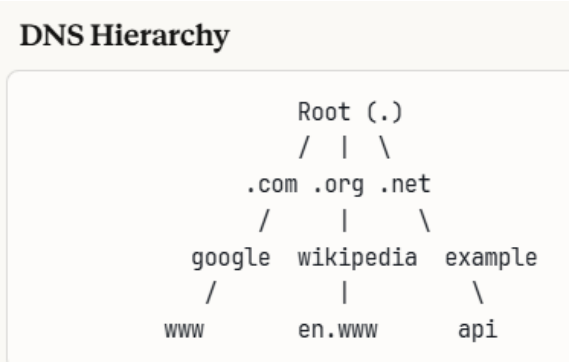
# Domain Name System

DNS is a distributed hierarchical database that translates human-readable domain names (like google.com) into IP addresses (like 142.250.80.46) that computers use to communicate.

**Key Interview Point**: DNS is essentially a **distributed key-value store** optimized for read-heavy workloads with hierarchical naming.

**DNS Hierarchy**




**Levels:**

1. **Root Servers** (13 globally, but replicated via anycast to hundreds of servers)
2. **TLD Servers** (Top Level Domain): .com, .org, .net, country codes
3. **Authoritative Name Servers**: Hold actual DNS records for domains
4. **Local DNS Resolver**: Your ISP or organization's DNS server (cache layer)

Anycast is a network addressing and routing methodology where **multiple servers share the same IP address**, and network routing automatically directs clients to the "nearest" server (in terms of network topology/routing).

**Key Principle**: One IP address → Many servers → Routing chooses the closest one

**Contrast with Other Routing Methods**

**Unicast (Traditional)**

One IP → One Server

Client → 192.168.1.1 → Always goes to Server A

**Broadcast**

One IP → All devices on network

Used in local networks (ARP requests, DHCP)

**Multicast**

One IP → Multiple subscribers who opt-in

Used for video streaming, stock tickers

**Anycast**

One IP → Nearest server in a group

Client → 1.1.1.1 → Goes to geographically/topologically closest server

**DNS Resolution Flow**

When you type www.example.com in your browser:

1. Browser Cache → check
2. OS Cache → check
3. Router Cache → check
4. If not cached, iterative query:

   Local Resolver → Root Server
   Root: "Ask .com TLD server at X.X.X.X"

   Local Resolver → .com TLD Server
   TLD: "Ask example.com's authoritative server at Y.Y.Y.Y"

   Local Resolver → example.com Authoritative Server
   Auth: "www.example.com is at Z.Z.Z.Z"

5. Local Resolver caches result and returns to client
6. Browser connects to Z.Z.Z.Z

**Why DNS is Designed This Way**

DNS is:

- ✅ **Distributed -** No single point of failure.
- ✅ **Hierarchical -** Makes global scaling possible.
- ✅ **Cached aggressively -** Reduces latency.
- ✅ **Highly available -** Root servers are globally replicated (anycast)

**Key DNS Record Types**

| Record | Purpose | Example |
|---|---|---|
| **A** | IPv4 address | example.com → 93.184.216.34 |
| **AAAA** | IPv6 address | example.com → 2606:2800:220:1:... |
| **CNAME** | Canonical name (alias) | www.example.com → example.com |
| **MX** | Mail exchange servers | example.com → mail.example.com (priority 10) |
| **NS** | Name server records | example.com → ns1.example.com |
| **TXT** | Text records | Used for verification, SPF, DKIM |
| **SRV** | Service location | _service._proto.name → host:port |
| **PTR** | Reverse DNS lookup | 34.216.184.93.in-addr.arpa → example.com |

## Load Balancing via DNS

**Round Robin DNS**: Return multiple A records in rotating order

dig example.com

ANSWER:

example.com. 300 IN A 192.168.1.1

example.com. 300 IN A 192.168.1.2

example.com. 300 IN A 192.168.1.3

**Limitations:**

- No health checking (may route to dead servers)
- Client-side caching issues
- Uneven distribution (some clients cache longer)
- TTL affects how quickly you can shift traffic

**Better Approach**: Use DNS to point to a load balancer, then load balancer handles health checks and distribution.

## Global Traffic Management (GTM)

**GeoDNS / Geo-routing**: Return different IPs based on client location

US client queries api.example.com → 192.168.1.1 (US datacenter)

EU client queries api.example.com → 192.168.2.1 (EU datacenter)

**Latency-based routing**: Return IP of closest/fastest datacenter

**Use Cases:**

- Reduce latency
- Comply with data residency requirements
- Disaster recovery

## TTL (Time To Live) Strategy

TTL determines how long DNS records are cached.

**Trade-offs:**

**Low TTL (60-300 seconds):**

- ✅ Fast failover/updates
- ✅ Quick traffic shifting
- ❌ Higher DNS query load
- ❌ More expensive
- Use for: Production systems needing fast updates

**High TTL (3600-86400 seconds):**

- ✅ Reduced DNS load
- ✅ Lower costs

- ✅ Better performance (fewer lookups)
- ❌ Slow changes/failover
- Use for: Stable infrastructure

**Interview Tip**: Mention you'd lower TTL before planned maintenance/migration, then raise it after.

## DNS for High Availability

**Multi-Region Setup**

Primary Authoritative Server (US-East)

↓ Zone Transfer

Secondary Authoritative Servers (US-West, EU, Asia)

**NS Records:**

example.com. IN NS ns1.example.com.

example.com. IN NS ns2.example.com.

example.com. IN NS ns3.example.com.

**Health Check-based DNS**

**Active-Active:**

Both datacenters receive traffic

Health checks remove unhealthy endpoints

**Active-Passive:**

Primary: api.example.com → 192.168.1.1

If primary fails, update to: api.example.com → 192.168.2.1

Wait for TTL expiration + propagation

**Problem**: DNS failover is slow (TTL + propagation)

**Solution**: Use Anycast or Application-level health checks with lower-level load balancers

## DNS Security Considerations

**DNSSEC (DNS Security Extensions)**

- Cryptographically signs DNS records
- Prevents DNS spoofing/cache poisoning
- Adds overhead but increases security

**DDoS Mitigation**

- DNS servers are common DDoS targets
- Use Anycast to distribute load
- Rate limiting
- Use managed DNS providers (Route53, Cloudflare, etc.)

**DNS Amplification Attacks**

- Attacker sends small query with spoofed source IP
- DNS server sends large response to victim
- Mitigation: Response rate limiting, disable recursion on authoritative servers

## Managed DNS Services

**AWS Route 53:**

- Health checks and failover
- Geo-routing, latency-based routing
- Integration with AWS services
- Anycast network

**Cloudflare DNS:**

- Fast global anycast network
- DDoS protection
- Free tier available

**Google Cloud DNS:**

- Low latency
- High availability
- Integration with GCP

## Common Interview Questions & Answers

**Q: How would you design a system to minimize downtime during datacenter failover?**

- Use low TTL (60-300s) for critical services,
- Implement health checks at DNS provider level,
- Use multiple NS servers across regions,
- Consider Anycast for instant failover,
- and have monitoring to detect issues before users do.

**Q: User in India is experiencing slow load times for your US-hosted app. How do you fix this?**

- Deploy application to region closer to India (Asia-Pacific),
- Implement GeoDNS/latency-based routing to direct Indian users to nearest datacenter,
- Use CDN for static assets, consider edge computing for dynamic content.

**Q: How do you handle DNS propagation during deployment?**

- Lower TTL 24-48 hours before deployment,
- Perform changes during low-traffic period,
- Use blue-green deployment where DNS gradually shifts traffic,
- Monitor error rates closely, have rollback plan ready, consider using weighted routing to gradually shift traffic (10% → 50% → 100%).

**Q: What's the difference between CNAME and A record? When would you use each?**

- A record maps directly to IP address (fast, one lookup).
- CNAME creates alias to another domain (requires additional lookup).
- Use A record for apex domain (example.com) and when you control IPs.
- Use CNAME for subdomains pointing to third-party services (CDN, load balancer) or when you want one canonical record to update multiple aliases.

**Q: What happens if DNS fails?**

- Entire service unreachable
- That's why companies use:
    - Multiple DNS providers
    - Short TTL
    - Health checks

**Q: Can DNS be attacked?**

Yes.

1. DNS cache poisoning
2. DDoS
3. DNS amplification attack

Solution:

- DNSSEC
- Rate limiting
- Anycast

---

**Scenario 1: Designing a Global Web App**

You say:

"We'll use DNS-based routing to route users to nearest region."

Example:

- User in Canada → AWS us-east
- User in India → AWS ap-south

This is called:

🌍 **GeoDNS**

Used by:

- Cloudflare
- AWS Route 53

**Scenario 2: Load Balancing**

DNS can do:

**Round Robin DNS**

app.com → IP1

app.com → IP2

app.com → IP3

Resolver returns different IPs.

⚠️ But:

- Not health-aware
- Not instant failover

So in real systems:
DNS → Load Balancer → App servers

**Scenario 3: High Availability**

DNS providers use:

**Anycast Routing**

Multiple global servers share same IP.

User automatically reaches nearest one.